

INDIAN INSTITUTE OF TECHNOLOGY, BOMBAY

B.TECH PROJECT REPORT

Automatic Short Answer Grading

Author:

Pratapa Venkata Adithya

Supervisor:

Pushpak Bhattacharyya
Shourya Roy

*A report submitted in fulfillment of the requirements
for the B.Tech project*

in the

Center for Indian Language Technology
Department of Computer Science and Engineering

May 2, 2017

Acknowledgements

I would like to thank Prof. Pushpak Bhattacharyya and Shourya Roy for introducing me to this amazing problem in the field of Natural Language Processing. This work would not have been possible without your constant support, technical guidance and insightful discussions. I would also like to thank the PhD students and staff at CFLIT lab, IIT Bombay for their help and support throughout this work.

Contents

Acknowledgements	iii
1 Automatic Short Answer Grading	1
1.1 Introduction	1
1.1.1 Automatic Evaluation	1
1.1.2 Essay Grading	1
1.1.3 Short Answer Grading	1
1.1.4 Problem Definition	1
1.1.5 Examples	2
1.1.6 Applications of ASAG	2
1.2 Prior Work	2
1.2.1 Intern Work	2
Key Ideas	2
Shortcomings	3
2 Approaches	5
2.1 Similarity	5
2.1.1 Lexical	5
2.1.2 Syntactic	6
2.1.3 Semantic	6
2.2 Paraphrasing	7
2.2.1 Recognition	7
2.2.2 Generation	8
2.2.3 Extraction	8
2.3 Textual Entailment	8
3 Case Studies	11
3.1 Lexical Measures	11
3.2 Syntactic Measures	11
3.3 Semantic Measures	12
3.4 Discussion	13
4 Universal Networking Language	15
4.1 Introduction	15
4.2 UNL Expression	15
4.2.1 Relations	15
4.2.2 Universal Words	16
4.2.3 Attributes	16
4.3 UNL vs Dependencies	17

5	Similarity Module	19
5.1	UNL System	19
5.2	Similarity	19
5.2.1	BLEU	19
5.2.2	Dependency Parse Tree:	19
5.2.3	Jiang and Conrath (JCN)	20
5.2.4	UNL	20
5.3	Evaluation	21
5.3.1	MSR Paraphrase Corpus	21
5.3.2	Computer Science Data set (ASAG)	21
5.4	Results	21
6	Discussion	23
6.1	Analysis	23
6.1.1	MSR Paraphrase Corpus	23
6.1.2	CSD	23
6.2	Issues	24
6.2.1	MSR Paraphrase corpus	24
6.2.2	CSD	25
6.3	Improvements	27
6.4	Conclusion	27
7	Word Embeddings	29
7.1	Introduction	29
7.1.1	GloVe	29
7.1.2	word2vec	29
7.2	Similarity	30
7.2.1	Cosine	30
7.2.2	Angular	30
7.2.3	Jaccard	30
7.2.4	Dice	30
7.3	Results	30
7.4	The New Baseline	31
8	Question Answering	33
8.1	Inspiration	33
8.1.1	Key concepts	33
8.1.2	Sequential pattern mining	33
8.2	Description	34
8.2.1	General framework of QA systems	34
8.2.2	Readings	34
8.3	Relevance to ASAG	35
8.3.1	Answer sentence selection	35
8.3.2	Sentence annotation	35
8.3.3	Answer extraction	35

9	A New ASAG system	37
9.1	Pattern vectors	37
9.2	Utilizing the question	37
9.3	Support	37
9.4	Relevance	38
9.5	Case Studies	39
9.6	Results	40
10	Question based evaluation	43
10.1	Inspiration	43
10.2	Graesser's taxonomy	43
10.3	Category based computation	44
10.4	Case Studies	44
10.5	Conclusion	47
11	Dependency based Word embeddings	49
11.1	Introduction	49
11.1.1	Paper: 'Dependency based word embeddings'	49
11.2	Relevance to ASAG	49
11.2.1	Results	50
11.3	Case Studies	50
11.4	Inference	52
12	Conclusion & Future Work	53
	Bibliography	55

Chapter 1

Automatic Short Answer Grading

1.1 Introduction

1.1.1 Automatic Evaluation

Evaluation of student's learning is an important part of every class room setting. This could be done in various ways, some of these use multiple choice questions, and some expect a natural language response. Evaluating the knowledge of student is better through natural language responses than MCQ's, but the task is easier in the latter. Manually evaluating the responses is difficult and time consuming. Thus, there is a need for automatic evaluation of these natural language responses.

1.1.2 Essay Grading

This problem is similar to short answer grading but works typically on larger responses, varying from few paragraphs to pages. This task differs from short answer grading both in the applications perspective and also the nature of evaluation. In essay scoring we broadly look into the writing style, grammatical correctness whereas in short answer scoring we look in for precise expected answer. We will later look into more details on the applications of these scoring techniques.

1.1.3 Short Answer Grading

Automatic short answer grading (or scoring) systems are designed to automatically evaluate 'short' answers presented in natural language. The general size of these answers vary from few phrases to a small paragraph. Some survey on the problem of short answer grading (Roy and Deshmukh., 2015), (Burrows and Stein., 2015).

1.1.4 Problem Definition

Our problem is to build a system that could automatically evaluate student answers given the question, reference answer and the set of student answers. The evaluation of the system is done by comparing the predicted scores with the human assigned scores using a correlation measure, in our case Pearson correlation measure. The need for system to understand the natural language and evaluate various key aspects of the answer makes it a challenging problem.

1.1.5 Examples

Given below is an example for short answer grading tasks.

Question: How are overloaded functions differentiated by the compiler?

Reference Answer: Based on the function signature. When an overloaded function is called, the compiler will find the function whose signature is closest to the given function call.

Student Answer 1: It looks at the number, types, and order of arguments in the function call.

Student Answer 2: By the number, and the types and order of the parameters.

We need to evaluate these student answers using the reference available to us.

1.1.6 Applications of ASAG

In the present day world of massive open online learning through various educational services like Coursera, Udacity etc., we have a large set of courses moving online from various top universities. These are followed by people all over the world and have gained a huge user base. In such large scale applications, it's always difficult to evaluate the student's understanding of the concepts through subjective methods. Automatic grading techniques would help accelerate this process and leads to better and more consistent assessment. Some of the currently applications of automatic evaluation, in the evaluation of GRE analytical writing where automatic essay scoring techniques are used to compound human judgments.

1.2 Prior Work

Here I briefly present the work I had previously done in short answer grading.

1.2.1 Intern Work

I started working on short answer grading during my intern at Xerox Research Center, India. The work here motivated me to continue it as my BTP. In following sections, I will briefly describe our work and the drawbacks of our system.

Key Ideas

We used a right combination of lexical, knowledge-based and vector spaced similarity measures basing on the knowledge of question's category. Our work is novel in designing strategies based on the question categories adopted from Graesser's question taxonomy. To overcome various limitations of traditional similarity-based approaches, we designed a sequential pattern mining technique to evaluate questions with high diversity in student answers. With this new technique, in addition to considering the provided reference answer, we also exploit the commonalities in student answers. This works as a feedback to the reference answer provided by the teacher to improve the

grading procedure. This approach is unsupervised and hence easily adaptable to various domains. We have improved the Pearson's correlation coefficient by 20% over the state of the art unsupervised techniques.

Shortcomings

We have tested our model only a single data set, a computer science question-answer set with 87 questions and approximately 30 answers for each question. We wanted to test the accuracy of this work on other short answer scoring data sets to develop a generic model. Some parts of our work, specifically the diversity annotation and partly the question categorization are done manually, we would like to automate the entire process to make it a truly unsupervised system.

Chapter 2

Approaches

In this work, we evaluated various strategies for the short answer grading task ranging from similarity at various levels, paraphrasing, textual entailment. We implemented a similarity module using UNL and the specific details are mentioned in chapter 5.

2.1 Similarity

Textual similarity is an active area of research in NLP and it has direct applications in major fields such as machine translation, sentiment analysis, summarization and question answering. We explored textual similarity at different NLP layers namely lexical, syntactic and semantic.

2.1.1 Lexical

We have different word based similarity measures namely,

1. **Edit distance:** The minimum number of transformations required to convert one word into another. Possible transformations are insertions, deletions and substitutions.
2. **Bag of words:** Sentences are viewed as a bag of words and the similarity is computed using these set of words. Some bag of words similarity measure relevant to our task are, Jaccard and Dice.
3. **Vector based:** Every word in the vocabulary is represented by a vector and similarity of the words is computed as a similarity of word vectors. There are different ways of generating or expressing word as a vectors. One easy way is to use one-hot representation which represents the words in the sentence as a vector of size of vocabulary and use cosine measure to measure similarity between the sentences. Other famous systems are Google word2vec and GloVe. When dealing with documents we can also use term frequency (TF) and inverse document frequencies (IDF).
4. **BLEU:** Bilingual evaluation study (BLEU) (Papineni, 2002) is primarily developed as an evaluation metric for automated machine translation. This comes under the Bag-of-words models. The measure is as give below,

$$p_n = \frac{\sum_{C \in \text{Candidates}} \sum_{n\text{-gram} \in C} \text{Count}_{clip}(n - \text{gram})}{\sum_{C' \in \text{Candidates}} \sum_{n\text{-gram} \in C'} \text{Count}_{clip}(n - \text{gram})} \quad (2.1)$$

$$BP = \begin{cases} 1 & \text{if } c \geq r \\ e^{(1-\frac{r}{c})} & \text{if } c \leq r \end{cases} \quad (2.2)$$

$$BLEU = BP \cdot e^{\sum_{n=1}^N w_n \log p_n} \quad (2.3)$$

Similar to the way BLEU is applied on candidate translation and reference translation, it can also be directly applied to find similarity between sentences and is used as a lexical baseline in our similarity module (refer to chapter 5).

2.1.2 Syntactic

Syntactic structure of the sentences can be represented by dependency parse tree and constituency parse tree.

1. **Dependency parse tree:** These parse trees are build using dependency grammars and all the nodes corresponds to words in the sentence and grammatical relationships are indicated by the connected edges.
2. **Constituency parse tree:** The words in the sentences are located at the leaf nodes and the non-terminals correspond to different grammatical categories such as noun phrase, verb phrase etc.
3. **N-gram stop word:** computes the common set of n-gram stop words between the sentences. These are useful in scenarios where capturing the stop word structure could be useful to finding the similarity. Some works have used these for plagiarism detection (Stamatatos, 2011).
4. **POS trigram:** computing the ratio of unseen trigrams of one text from the other (Kong, 2014).

In this work, we use dependency parse as a part of syntactic similarity computation.

2.1.3 Semantic

Unlike lexical and syntactic measure, semantic measures uses the meanings of the words (or tokens) in the sentences. WordNet is used to find the semantic similarity in the following measures,

WordNet: An online lexical reference system with English nouns, verbs, adjectives and adverbs organized in synsets (synonym sets). It is used to capture semantic relations between words and the similarity between words can be captured in many different as follows:

1. **Shortest Path:** distance between the two concepts in WordNet. The farther they are in the WordNet, lesser is the similarity. *length* is the distance between the concepts in WordNet.

$$Similarity = \frac{1}{length(c_1, c_2)}$$

2. **Resnik:** Resnik., 1995

$$Similarity = IC(LCS(c_1, c_2))$$

3. **Leacock & Chodorow:** Leacock and Chodorow., 1998

$$Similarity = -\log \frac{length}{2 * D}$$

4. **Wu Palmer:** Wu and Palmer., 1994

$$Similarity = \frac{2 * depth(LCS)}{depth(concept_1) + depth(concept_2)}$$

5. **Jiang and Conrath:** Jiang and Conrath., 1997

$$JCN(c_1, c_2) = \frac{1}{IC(c_1) + IC(c_2) - 2 * IC(LCS(c_1, c_2))}$$

In the above formula, IC stands for the information content and LCS stands for least common subsumer.

In this work, we have used Jiang and Conrath (JCN) as a semantic baseline to our system. In section 4, we will look into a deep similarity measure which is a key part of our similarity module.

2.2 Paraphrasing

Description: The basic task is to evaluate the paraphrase relation in a pair of sentences. There are three different versions of the problem namely (Androutsopoulos and Malakasiotis., 2010),

2.2.1 Recognition

The task here is to check for paraphrase or textual entailment relationship between a given pair of sentences. These techniques can also help in recognizing relations between given reference answer and the student answer.

1. **Logical meaning representation:** paraphrasing relation with the problem of logical entailment is explored.
2. **Vector space models:** Language expressions are mapped into vectors and similarity measures like cosine similarity are used to test the paraphrase relationship.
3. **Surface string similarity:** The input strings are first pre-processed and then string similarity measures like string edit distance, BLEU, sliding window, sliding alignment are used. Word alignment methods are used for entailment.

4. **Syntactic similarity:** Tree edit distance on dependency parse tree with appropriate costs, or some tree similarity measures. These are better than surface similarity measures in capturing relations. For example, consider the following text and hypothesis,
 T: The National Institute for Psychobiology in Israel was established in 1979
 H: Israel was established in 1979
 But it mainly depends on the robustness of parsers and sometimes string based measures could perform better.
5. **Similarity measures on symbolic meaning representations:** semantic relations are explored, for example WordNet synonyms, hypernyms-hyponyms etc.
6. **Machine learning:** similarity levels are combined with machine learning techniques and they can also be used to generate features.
7. **Decoding:** a sequence of rule applications, for example "X is fond of Y" $\sim$ "X likes Y"
8. **Data sets:** one well known data set for the task of paraphrase detection is, Microsoft research paraphrase corpus consisting of 5801 pairs of sentences. RTE challenges are also regularly held to promote and advance the research in the field of textual entailment.
9. **Evaluation:** paraphrasing and textual entailment techniques are generally evaluated by computing various measures like accuracy, precision, recall and f-measure.

2.2.2 Generation

The problem here is to generate paraphrases for a given input sentence. Though generation task is not directly useful for our task of automatic short answer grading (ASAG) but it could be useful in generating more references answers, which could lead to better evaluation of the student answers. In the short answer data set we use for evaluating our similarity module in this work, every question has only reference answer but a set of 30 student answers.

2.2.3 Extraction

We have to extract paraphrases for a given input sentence from a corpus or web. This is also not directly connected to our problem, but it could be used for extracting relevant reference answers or to retrieve text content from student answer that is most relevant to the reference answer. Finding a text corpus to extract paraphrases could be a limiting factor here.

2.3 Textual Entailment

Description: Given a piece of information (text, t), can a human most likely infer the hypothesis (h). Representing this as a logical expression, $t \implies h$. Paraphrasing can be thought as bidirectional textual entailment. Similar to paraphrasing we have also

have recognition, generation and extraction tasks relating to textual entailment. Techniques used for textual entailment are only slight modifications of the paraphrasing techniques.

Chapter 3

Case Studies

In this chapter, we present a study of various similarity measures and some insights on which similarity measures are more suitable to short answer grading.

3.1 Lexical Measures

The lexical measure we use in this work is BLEU score. This is also a bag of words similarity measure. Consider two sentences as given below,

Sentence 1: *The teacher awarded the student.*

Sentence 2: *The student awarded the teacher.*

Bag of words cannot distinguish between these two types of sentences since the set of words are the same along with their tense but their roles in the sentence are different. Thus in most of the cases, there is a need to look at similarity measures that also captures the structure of the sentences. Note that BLEU cannot detect this if we just consider 1-grams but can distinguish for higher n-grams.

3.2 Syntactic Measures

Dependency parse trees and constituency parse trees can be used to capture the grammatical structure of the sentences. For example, dependency parse can distinguish above sentences 1 and 2¹, the subject relation maps to different pairs of words in these sentences. Consider a new set of sentences as given below,



FIGURE 3.1: Basic dependencies of sentence 1

Sentence 3: *The student was awarded by the teacher.*

Sentence 4: *The teacher was awarded by the student.*

¹These images are generated using stanford parsers available at www.corenlp.run

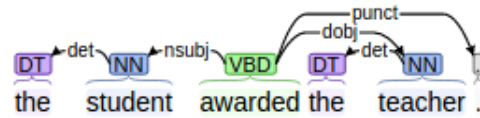


FIGURE 3.2: Basic dependencies of sentence 2

These two sentences carry different meanings but sentence 3 is semantically same as the sentence 1 and sentence 4 is semantically same as the sentence 2. These are also captured by the basic dependency relations using the 'passive auxiliary' (auxpass)

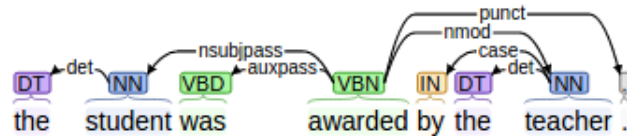


FIGURE 3.3: Basic dependencies of sentence 3

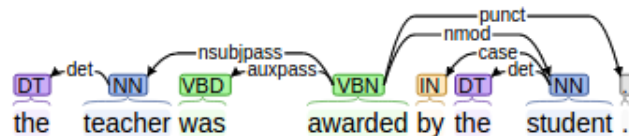


FIGURE 3.4: Basic dependencies of sentence 4

3.3 Semantic Measures

In this section, we discuss the effectiveness of semantic measures to capture the relations between sentences. We consider UNL as our deep semantic similarity measure. The exact details of UNL are explained in the next chapter. All the UNL expressions used in this section are generated using the English-UNL enconverter module, a part of the ETAP-3 linguistic processor developed in the Computational Linguistics Lab of the Institute for Information Transmission Problems of the Russian Academy of Sciences. Consider the UNL expressions for sentences 1 and 3,

```
[S:00]
{org:en}
The teacher awarded the student.
{/org}
{unl}
agt(award(icl>present>do,agt>thing,obj>thing,rec>thing).@entry.@past,
teacher(icl>educator>thing).@def)
obj(award(icl>present>do,agt>thing,obj>thing,rec>thing).@entry.@past,
student(icl>university_student>person,obj>knowledge_domain).@def)
```

```
{/unl}
[/S]
```

```
[S:00]
org:en
The student was awarded by the teacher.
/org
unl
obj(award(icl>present>do,agt>thing,obj>thing,rec>thing).@entry.@past.@passive,
student(icl>university_student>person,obj>knowledge_domain).@def.@topic)
agt(award(icl>present>do,agt>thing,obj>thing,rec>thing).@entry.@past.@passive,
teacher(icl>educator>thing).@def)
/unl
[/S]
```

Note that UNL can also distinguish the sentences 1 and sentences 3 but student remains the object and thus matching the relations and corresponding universal words would give a good similarity score. If we compare the UNL document for sentence 3 and sentence 4 (given below), we can see that the universal words corresponding to the *obj* relation have changed.

```
[S:00]
org:en
The teacher was awarded by the student.
/org
unl
obj(award(icl>present>do,agt>thing,obj>thing,rec>thing).@entry.@past.@passive,
teacher(icl>educator>thing).@def.@topic)
agt(award(icl>present>do,agt>thing,obj>thing,rec>thing).@entry.@past.@passive,
student(icl>university_student>person,obj>knowledge_domain).@def)
/unl
[/S]
```

3.4 Discussion

From the above experiments we observed that bag of words isn't an ideal measure because it fails to capture the grammatical structure of the sentence and also the meanings of words in the sentence. We also noted that both basic dependency relations and UNL are able to capture major differences like subject of the sentence and also subtle differences like voice of the sentence, thus can be an accurate measure for textual similarity than lexical measures.

But the important question we have here is whether UNL captures more than dependency parsing in the context of textual similarity? This can be better understood if go in more detail about the working of Universal networking language. In the following chapter, we describe various components of UNL document and significance of each of its parts.

Chapter 4

Universal Networking Language

Universal Networking Language (UNL) ^{1 2} is a computer language that enables computers to process information and knowledge across the language barriers. The UNL project is an initiative by Institute for Advanced Studies and United Nations University. It is one of the well used inter-lingua and is considered to be completely unambiguous representation of natural languages.

4.1 Introduction

UNL is used to represent information and knowledge provided by the natural languages. It retains all the natural language components through various entities namely universal words, relations and attributes.

4.2 UNL Expression

The information is expressed as a semantic network over hyper-nodes. This semantic network with nodes representing the concepts and edges denoting the relations between these concepts is unambiguous and is termed as "UNL expression". Sentences are represented by UNL document which in turn consists of UNL expressions. All the parts of natural language, paragraphs, sentences form semantic network over hyper-nodes. UNL expressions have the following form,

$$\langle \text{relation} \rangle \ (\langle \text{uw1} \rangle, \ \langle \text{uw2} \rangle)$$

The UNL semantic network is a directed graph and some relations capture the essence of this direction. Relations can also operating on hyper-nodes themselves, in such case we define a scope id to label a particular node.

$$\langle \text{relation} \rangle : \langle \text{scope-id} \rangle \ (\langle \text{node1} \rangle, \ \langle \text{node2} \rangle)$$

4.2.1 Relations

UNL relations form describe the directed semantic functions between universal words. Conceptually similar but very different grammatical structures are distinguished by these relations. There are a total of 57 relations and listing them categorically,

1. **Case relations:** Connecting actions, events or states with things (agent, manner, place, role etc.,)

¹<http://www.undl.org/unlsys/unl/>

²<http://www.unlweb.net/wiki/Specs>

2. **Inter matter relations:** Causal, purpose relations among actions, events or states (condition, co-occurrence, purpose, sequence etc.,)
3. **Restrictive relations:** (content, modification, member of, proportion, part of etc.,)
4. **Grouping relations:** (and, intersection, disjunction)
5. **Inclusion relations:** (equivalence, included in, instance of)
6. **Compound relations:** can be replaced by corresponding UW (possessor, author, origin, destination etc.,)
7. **Structural relations:** (sentence head)

4.2.2 Universal Words

These form the nodes of the semantic network of UNL and constitute the vocabulary of UNL. The generate structure of universal word is,

`<UW> ::= <headword> [<constraint list>]`

Constraint lists can in turn consists of relations, universal words and sub constraint lists.

4.2.3 Attributes

These relate the node with itself and are used to represent the natural language grammatical categories. Listing different types of attributes with examples,

1. **Logicity of UWs:** @transitive, @symmetric
2. **Times with respect to speaker:** @past, @present, @future
3. **Speaker's view of aspects of events:** @begin, @continue, @repeat
4. **Speaker's view of reference to concepts:** @generic, @def, @etc, @not
5. **Speaker's view of emphasis, focus and topic:** @contrast, @emphasis
6. **Speaker's attitude:** @affirmative, @imperative, @request
7. **Speaker's feelings and judgments:** @ability, @conclusion, @consequence
8. **Orders:** @first, @last
9. **Relative degrees:** @less, @least, @more, @most
10. **Convention** @passive, @pl, @paranthesis

4.3 UNL vs Dependencies

Dependencies capture the grammatical relations in the sentence whereas UNL captures the conceptual relations. Conceptual relations try to relate very different grammatical structures which are conceptually the same. For example, *agent* is a semantic concept and is distinct from subject of the sentence. Consider the two sentences, "*The teacher awarded the student*", "*The student was awarded by the teacher*". In this case *agent* is the same, grammatical subject changes (syntactic, depends on word order).

Unlike dependencies, UNL also captures the semantics of the sentence. This is achieved through a combination of it's relations, universal words, attributes and constraints. UNL is generally used as interlingua in Machine Translation systems where a source language text can be en-converted to UNL and then de-converted to target language. This could be done because UNL is considered to be unambiguous.

Consider the following examples where UNL differs from typed dependency relations, *Semantic dependencies than syntactic roles:*³

David received the book (*gol*, specifier)

Amy gave the book to David (*gol*, complement)

Lexical and syntactic ambiguities

David works in Chicago (*plc*)

David works in summer (*tim*)

Internal semantic structure

David kills Amy (subject(*agent*), object(*patient*))

David loves Amy (subject(*experiencer*), object (*content*))

³http://www.unlweb.net/wiki/Universal_Relations

Chapter 5

Similarity Module

In this section, we present the similarity module we have developed primarily for the task of Automatic short answer grading (ASAG). After getting insights from experimenting with similarity measures of different levels as presented in chapter 3, we designed similarity measures of different levels (lexical, syntactic and semantic) and evaluated their performance for our short answer grading task.

5.1 UNL System

Main part of our similarity module works on the UNL graphs of the input answers and to generate these UNL expressions we have used the English-UNL en-converter module developed as a part of ETAP-3 linguistic processor in the Computational Linguistics Lab of the Institute for Information Transmission Problems of the Russian Academy of Sciences. We thank Prof. Igor Boguslavsky and his team for providing the system for our research purposes.

5.2 Similarity

We have used four different similarity measures, namely, BLEU (lexical), dependency parse tree matching (syntactic), JCN and UNL (semantic).

5.2.1 BLEU

We have adopted the BLEU (Bilingual evaluation understudy) implementation from python natural language toolkit (NLTK).

Pre-processing of the text removes stop words, punctuation and lemmatizes using *WordNetLemmatizer* from NLTK stem module. The computation of BLEU is same as present in earlier chapter and we have only considered uni-grams in computing the BLEU score.

5.2.2 Dependency Parse Tree:

We have used the Stanford PCFG parser (Klein and Manning., 2003) to generate grammatical relations between the words in our text. Only the typed dependencies are used as a part of similarity computation.

1. **Relational similarity:** We check for one on one match between words for a given pair of relations with same name. Because the grammatical relations are directed, we match the first word in first relation with the first in second, and similarly the second word with second word. We also using *WordNetLemmatizer* to lemmatize the words to their base

form to account for better matching.

$$word_sim = \begin{cases} 1.0, & \text{if } w_1^1 = w_1^2 \text{ \& } w_2^1 = w_2^2 \\ 0.0, & \text{otherwise} \end{cases} \quad (5.1)$$

2. **Sentential similarity:** A sentence level, we find the common set of relations between these sentences and for each such relation we find the best match in the other sentence. We average this score over the entire set of common relations.
3. **Document similarity:** Using the dependency parser each sentence in our document is represented in terms of grammatical relations and words associated with these relations. For every sentence in one document, we find the best match sentence in the other by matching the corresponding relations as given by the above similarity measure. The score is then averaged over the number of sentences in the document.

5.2.3 Jiang and Conrath (JCN)

This similarity measure was proposed by Jiang and Conrath and given similarity between two concepts c_1 and c_2 is given as follows,

$$JCN(c_1, c_2) = \frac{1}{IC(c_1) + IC(c_2) - 2 * IC(LCS(c_1, c_2))}$$

Information content, $IC(c) = -\log(P(c))$ and $LCS(c_1, c_2)$ stands for least common subsumer, i.e the most specific ancestor node to c_1 and c_2 in the WordNet.

5.2.4 UNL

As mentioned earlier, the UNL documents are generated using the English-UNL enconverter module. The implementation of this similarity measure is partly similar to that of dependency parsing based match. Some prior approaches on using UNL for textual similarity Dan and Bhattacharyya., 2013, Gautam and Bhattacharyya., 2014.

1. **Universal word similarity:** Universal words consists of the base word, attributes and the constraints. We have separately matched each and weighted them to get a overall measure as given below

$$uw_sim = 0.5 * (word_match) + 0.5 * (word_match) * (attribute_match) * (constraint_match)$$

2. **Relation similarity:** UNL relations with same name are matched individually in the universal words associated with the relation. Note that the UNL relations are directional.

$$rel_sim = 0.5 * uw_sim(rel1.words[0], rel2.words[0]) + 0.5 * uw_sim(rel1.words[1], rel2.words[1])$$

3. **Sentence similarity:** Given a pair of sentences, we find the common set of UNL relations and for each such relation we find the best match from the other sentence. We average this score over the size of common relations.
4. **Document similarity:** Similarity between documents are in a manner similar to sentence similarity. Every document consists of a set of sentences and for every sentence we find the best matching sentence from the other document. Best matching in all the above scenarios mean the one with maximum similarity score.

5.3 Evaluation

Though our aim is to develop similarity measures to evaluate answers, we also tested the similarity measures on Microsoft research paraphrase corpus.

5.3.1 MSR Paraphrase Corpus

MSR Paraphrase corpus (Quirk and Dolan., 2004) consists of 5801 pairs of sentences collected from thousands of news articles over the web. Each pair is human annotated with a score of 0 or 1, denoting the paraphrase relationship between the sentences. We have used 94 sentence pairs from this dataset for evaluating each of our similarity measures.

5.3.2 Computer Science Data set (ASAG)

This dataset was provided by Mohler and Mihalcea., 2011, Mohler and Mihalcea., 2009 and is a short answer grading dataset consisting of 87 questions, with one reference answer and approximately 30 student answers for each question. Two human annotators have independently annotated all the answers, we have considered the average of these two scores as the gold standard score.

5.4 Results

We will separately present the results of various similarity measures on different datasets and discuss the results in more detail in the upcoming chapters. In both the cases we have reported Pearson's correlation coefficient between the predicted output to the human annotated output.

1. MSR Paraphrase Corpus

TABLE 5.1: Pearson r values on MSR Paraphrase data set

Similarity measure	No. of sentence pairs	Pearson r
BLEU	94	0.56
Dependency	94	0.42
JCN	94	0.43
UNL	94	0.38

TABLE 5.2: Similarity values on MSR Paraphrase data set

Similarity measure	Positive			Negative		
	Mean	Minimum	Maximum	Mean	Minimum	Maximum
BLEU	0.63	0.36	0.92	0.45	0.22	0.76
Dependency	0.50	0.06	0.85	0.30	0.0	0.80
JCN	0.52	0.18	0.80	0.37	0.0	0.86
UNL	0.39	0.13	0.76	0.26	0.0	0.62

TABLE 5.3: Precision, recall and F-measure on MSR Paraphrase data set

Similarity measure	No. of sentence pairs	Precision	Recall	F-measure
BLEU	94	0.79	0.79	0.79
Dependency	94	0.89	0.57	0.69
JCN	94	0.79	0.59	0.67
UNL	94	0.88	0.26	0.4

2. Computer science dataset (CSD)

In the computer science, some student answers are large and had problems while generating the dependency relations. Similarly, the UNL system had problem with few answers while generating UNL expressions. Such answers are excluded and Pearson r is reported.

TABLE 5.4: Pearson r values on CSD

Similarity measure	No. of questions	Pearson r
BLEU	87	0.36
Dependency	61	0.28
JCN	87	0.45
UNL	60	0.35

Chapter 6

Discussion

The results using various similarity measures both on the paraphrase data set and computer science data set are presented in previous chapter. Here we are going to discuss the significance of the results and analyze the reasons on why some techniques performed the way they did.

6.1 Analysis

Let's discuss the performance separately on each of the data set. This is required because the text format and scoring techniques are very different for these data sets. MSR Paraphrase corpus has sentence pairs each of which is a single sentence and output is either 0 or 1, '1' if they are paraphrases, '0' otherwise. But the computer science data set consists of answers which typically consist of few sentences and the score allotted for these answers are in the range zero to five (in 0.5 steps). The nature of sentences are also very different in these data sets. In general, sentences pairs in Paraphrase corpus are more related to each other than reference and student answer pairs in CSD.

6.1.1 MSR Paraphrase Corpus

Pearson r scores are best for BLEU similarity measure followed by JCN, dependency and UNL. The correlation scores are very similar for dependency and JCN but are substantially low for UNL. Pearson r is not a quite apt measure for paraphrase recognition task because the annotated scores are either 0 or 1 and correlation coefficient works better when we have varied values of scores, for example as in short answer grading task. Thus we also reported precision, recall and f-measure for paraphrase detection task.

Precision scores are generally high and are similar for all the similarity measures whereas recall differs a lot from one similarity to other. Though precision is high for the UNL similarity measure, it's recall is the least (just 0.26) and thus reflected in the low f-measure.

Lexical measures have the best recall value followed by syntactic measures and then semantic measures. Note that we can directly compare between these similarity measures since we used the same set of 94 pairs in all the computations.

6.1.2 CSD

Pearson correlation is a widely used evaluation measure for the task of automatic short answer grading. The pearson r values for each similarity measure are reported in the table 5.4. As stated earlier, all the set of 87 questions were used for BLEU and JCN similarity measures but only a subset of them are used for dependency and UNL measures because of limitations of the system used to generate dependency relations and UNL expressions.

JCN performed the best by a large margin with UNL and BLEU performed similarly and a very low score for dependency relations based matching.

In the upcoming section we will discuss the reasons for sub standard performance for some similarity measures.

6.2 Issues

6.2.1 MSR Paraphrase corpus

On this data set the performance of UNL system isn't great, though it's comparable to JCN and dependency in terms of pearson r, BLEU performed way better. But as discussed earlier, pearson r is not a apt measure for to evaluate the performance on this data set, we consider the performance in terms of statistical measures like precision, recall and f1-score.

One preliminary observation from the table 5.2, is that in general the mean similarity score of for positive examples is higher than the mean score for negative examples. This is expected as the sentences pairs which share a paraphrase relationship are similar than the ones which aren't paraphrases. But all the similarity measures generated similarity scores over a wide range, for example, the range of BLEU similarity scores for positive examples is (0.36, 0.92) and for negative examples is (0.22, 0.76). Thus there is not distinct boundary where we can directly separate the positive and negative examples. But we considered '0.5' similarity score as our boundary, below which the sentences don't form a paraphrase pair.

But one positive result from the precision data is UNL has the best precision measure among the four similarity measures. It is significantly better than BLEU and JCN. But the precision is very low. So it is unable to detect all the positive examples, but when ever it detects it, is it mostly likely true. Unfortunately, this logic is not entirely correct.

From table 5.2 we can observe that, the average similarity scores using UNL are substantially lower than other measures both among positive and negative instances. Since, we have chosen our threshold to be 0.5 to compute precision and recall, only few of the UNL similarity scores are above this threshold and if they are above this, then it is actually very likely that they are paraphrases, hence the higher precision. But there are many false positives which have UNL similarity scores below 0.5 and hence the low recall.

Consider the following example,

Sentence 1: *A BMI of 25 or above is considered overweight; 30 or above is considered obese.*

Sentence 2: *A BMI between 18.5 and 24.9 is considered normal, over 25 is considered overweight and 30 or greater is defined as obese.*

These sentences are not a paraphrases because one expresses more information than the other. The similarity scores are,

BLEU: 0.54

Dependency: 0.46

JCN: 0.45

UNL: 0.15

At first glance UNL seems to perform better, but actually the relations in UNL expressions for these two sentences are way different because of different framing of the sentences. Though they capture the meaning unambiguously, the shallow relation match we perform here is responsible for the very low score.

6.2.2 CSD

The major difference we can observe is low pearson r scores on CSD data set than MSR paraphrase data set. This is expected because sentences constituting the paraphrases have majority of the text in common and have few additional text. These differences are easily captured by our lexical measures and syntactic measures.

Surprising results on CSD data sets are the performance of dependency relational matching and UNL graph matching. Since, these techniques capture grammatical structure and semantic meanings along with the word tokens, these are expected to do better than lexical baseline BLEU. For example, JCN also matches concept by concept but it captures meaning of the concept which isn't considered in BLEU measure. We find a significant difference in the performance of JCN (0.45) and BLEU (0.36).

On deeper analysis into the evaluation using UNL expressions, we found that the similarity scores generated by UNL measure are in general lower than the corresponding lexical measures. This is because at the top level we are considering universal relations and match these between sentences. These relations capture the semantic functions between the words and slight variation in the sentence could change these relations significantly because the UNL expressions should represent the natural language sentence without ambiguity.

We will illustrate this phenomenon with few example answers,

Sentence: *All of the dimensions*

```
per(all(icl>quantity, ant>some, per>thing) .@topic,
dimension(icl>magnitude>thing) .@def.@pl)
```

Sentence: *All the dimensions*

```
qua(dimension(icl>magnitude>thing) .@entry.@def.@pl,
all(icl>quantity, ant>some, per>thing))
```

Both the sentences have the same meaning but the relations capture similar but distinct meanings, proportion (*per*) and quantity (*qua*). Our similarity measures fails to capture this.

But there are examples where UNL performs better than BLEU in scoring,

Sentence: *All the dimensions, except the first one.*

```
qua(dimension(icl>magnitude>thing) .@entry.@def.@pl,
    all(icl>quantity, ant>some, per>thing))
man(dimension(icl>magnitude>thing) .@entry.@def.@pl,
    except(icl>how, obj>thing))
mod(one(icl>digit>abstract_thing) .@def,
    first(icl>adj, ant>last))
obj(except(icl>how, obj>thing),
```

```
one(icl>digit>abstract_thing).@def)
```

Sentence: *Just the first one at least.*

```
man:01(first(icl>how).@entry,
      just(icl>how,equ>merely,com>mere))
man:01(first(icl>how).@entry,
      at_least(icl>how))
man(one(icl>digit>abstract_thing).@entry.@def,
     :01)
```

Here, the student answer is wrong and our UNL measure gives a score of 0.0 but BLEU gives a score around 0.4. Thus relations are useful, possibly we aren't capture the entire information provided by UNL. This is clearly illustrated by the following examples,

Sentence 1: *All the dimensions, except the first one.*

```
qua(dimension(icl>magnitude>thing).@entry.@def.@pl,
    all(icl>quantity,ant>some,per>thing))
man(dimension(icl>magnitude>thing).@entry.@def.@pl,
    except(icl>how,obj>thing))
mod(one(icl>digit>abstract_thing).@def,
    first(icl>adj,ant>last))
obj(except(icl>how,obj>thing),
    one(icl>digit>abstract_thing).@def)
```

Sentence 2: *All but the first.*

```
man(all(icl>quantity,ant>some,per>thing).@entry,
    but(icl>how,equ>except_for,obj>uw,man<uw))
obj(but(icl>how,equ>except_for,obj>uw,man<uw),
    first(icl>adj,ant>last).@def)
```

Sentence 3: *All dimensions, excluding the first one.*

```
qua(dimension(icl>magnitude>thing).@entry.@pl,
    all(icl>quantity,ant>some,per>thing))
agt(exclude(icl>eliminate>do,src>uw,agt>thing,obj>thing).@progress,
    dimension(icl>magnitude>thing).@entry.@pl)
mod(one(icl>digit>abstract_thing).@def,
    first(icl>adj,ant>last))
obj(exclude(icl>eliminate>do,src>uw,agt>thing,obj>thing).@progress,
    one(icl>digit>abstract_thing).@def)
```

First one of the above three examples is the reference answer. We can see both the student answers given above are correct but UNL doesn't give high scores because of relational mismatch. Consider the 'man' and 'obj' relations in sentences 1 and 2, the universal words associated with these relations are very similar. Sentence 1 has man connecting 'dimension' and 'except' and in sentence 2 it connects 'all' and 'but'. But 'dimension' is connected to 'all' via relation qua in sentence 1, using this will help in capturing the relation. Similarly, universal word but has restrictions on it equ>except_for which helps in relating it with except in sentence 1.

Similar is the case with matching sentence 1 and sentence 3.

6.3 Improvements

From the case studies in the previous section, we find that though UNL is able to unambiguously capture the information from the sentence, we aren't completely utilizing this information. We thereby realize the need to look more into the UNL graph than just the shallow level relational matching.

6.4 Conclusion

From the analysis in last section, UNL is able to capture the information well but the current similarity measure is unable to reflect it. The current layered approach of matching relations at the top level, followed by universal words and attributes is not adequate and there is a need for re-considering relations while matching the UWs. Similarly, relations in the constraints should be better understood and captured rather than a string match. These constraints can sometimes be useful in detecting UWs which are related to each other though they are a part of entirely different set of relations. The UNL system after these modifications will possibly perform better than our baseline systems, BLEU, JCN and dependency parsing. This is because UNL covers all the aspects covered by above baselines like semantics of the sentence, relationships among words.

Other relevant problems to short answer grading like plagiarism detection should be explored. Deep learning can also be tried for ASAG and access various aspects of it's performance.

Chapter 7

Word Embeddings

7.1 Introduction

Words in the vocabulary are mapped into vectors of real numbers. There are different ways to generate these word embeddings using, neural networks, dimensionality reduction on word co-occurrence matrix and probabilistic models. These are of very significance in many natural language processing tasks like sentiment analysis, machine translation. For example, in this work we consider it's relevance to the similarity computation which is a key component of many NLP tasks.

Vector representations of words capture the contextual, semantic meaning of the words. One-hot vector is a way of representing each word of the vocabulary where the vector is of the vocabulary size, one entry is 1 denoting the word and others are all zero. This is a sparse representation and possibly of very high dimension. In practice the word embeddings are dense and low-dimensional (300) vectors.

Here we briefly describe a few famous word embeddings,

7.1.1 GloVe

This is an unsupervised way of obtaining word representations and the aggregated global word co-occurrence matrix is built using the corpus. This work is first presented in the work GloVe: Global Vectors for Word Representation (Pennington, Socher, and Manning., 2014). It exhibits linear substructure properties in the word vector space, for example man-woman, king-queen in the vector space are roughly equal. ¹

7.1.2 word2vec

This implementation uses skip-gram models with negative sampling with skip-gram models to generate continuous vector representations (Mikolov, 2013). It provides two learning algorithms, continuous bag-of-words and continuous skip-gram. Similar to GloVe (GloVe), word2vec also exhibits the linear substructure property and also capture the semantic relations among words via their continuous vector representations. ²

Though the final word representations generated from GloVe and word2vec perform similarly in many NLP tasks but the framework is different, with the former being a count-based model and latter being a predictive model. We will see the performance of both of these representations on our short answer grading problem.

¹<https://nlp.stanford.edu/projects/glove/>

²<https://code.google.com/archive/p/word2vec/>

7.2 Similarity

As discussed in chapter 2 (2), similarity computation between the student answers and the provided reference answer is a key approach for ASAG, here we look at a new and different perspective of computing this similarity.

We use the vector representations for words and generate a same dimensional vector representing the answers. We hypothesize that the average of word vectors of words in a given answer can denote the answer as a whole. After computing these vectors for each of the student answers and also the reference answers, we use the following vector based similarity measures to compute the scores.

7.2.1 Cosine

Cosine similarity between two non-zero vectors A and B is defined as follows:

$$\text{cosine_similarity} = \text{cosine}(\theta) = \frac{A.B}{\|A\|.\|B\|}$$

7.2.2 Angular

Angular similarity is similar to cosine and can be computed as follows:

$$\text{angular_similarity} = 1 - \frac{\cos^{-1}(\text{cosine_similarity})}{\pi}$$

7.2.3 Jaccard

Jaccard similarity between two non-zero vectors A and B is defined as follows:

$$\text{jaccard_similarity} = \frac{A.B}{\|A\|^2 + \|B\|^2 - A.B}$$

7.2.4 Dice

Dice similarity between two non-zero vectors A and B is defined as follows:

$$\text{dice_similarity} = \frac{2.A.B}{\|A\|^2 + \|B\|^2}$$

7.3 Results

We have utilized pre-trained word vectors on Google news dataset, provided by Google and it contains 300 dimensional word vectors for 3 million words and phrases. Utilizing the word representations, we computed an average vector denoting each of the reference and student answers. These vectors are then compared using the similarity measures presented in earlier section to generate scores for student answers.

Below tables present these results on the computer science dataset consisting a total of 87 questions with approximately 30 student answers for each. We have reported both the pearson r score and the root mean square error between the human annotated scores and system generated scores.

TABLE 7.1: Pearson r and RMSE values for different vector based similarity measures on CSD dataset (word2vec)

Similarity measure	No. of questions	Pearson r	RMSE
Cosine	81	0.501	1.76
Angular	74	0.430	1.21
Jaccard	81	0.479	2.46
Dice	81	0.497	1.84

We also generated the results using pre-trained GloVe vectors,

TABLE 7.2: Pearson r and RMSE values for different vector based similarity measures on CSD dataset (GloVe)

Similarity measure	No. of questions	Pearson r	RMSE
Cosine	81	0.462	1.40
Angular	77	0.385	1.12
Jaccard	81	0.450	2.00
Dice	81	0.458	1.44

7.4 The New Baseline

The cosine distance based similarity measure used on the word embeddings have given a new best pearson r value of 0.501. Interestingly, is was obtained using a simple algorithm and thus signifies the power of word embeddings. These embeddings are trained a huge corpus and capture the semantics of these words based on the contexts. We thus use word embeddings instead of words for the upcoming works and this would serve as the new baseline.

Chapter 8

Question Answering

Systems that automatically answer natural language questions posed by humans. It is one of the leading problems in Natural language processing with roots as early as in 60s and 70s. Question answering is similar to our problem of short answer grading but not the same. Here, we have to grade the answer given both the question and answers whereas in QA, we try to find the best possible answer to the given question. Many QA systems focus on answering factual questions which is very different from dealing the questions in our computer science dataset where majority are reasoning or opinion based.

8.1 Inspiration

Exploring QA is partly motivated by it's slight resemblance with ASAG, but more concrete reasons are presented below,

8.1.1 Key concepts

Similarity can be used as a technique for short answer grading but there are innate difference between the requirements of these two problems. In similarity we look for how close the two texts are whereas in the framework of short answer grading we need to evaluate the student answer given the question and reference answer. The student answer being as close to reference answer as possible definitely helps but the main requirement is to present the necessary aspects (or concepts) in the student answer. The reference answer should just be used as a guideline.

Key concepts are the sentences or phrases which correspond to the correct answer and there could many such potential phrases. It might not be the case that the reference answer covers every one of them. Thus the identification of these concepts from the question, reference answer and the student answer corpus is a key step to the problem of short answer grading.

Question answering ideas could be helpful for identifying the key concepts. In QA, we are just provided with the question and we have to search the entire corpus to find the right answer. This is a way to identify the key phrases relating to the given question from the corpus.

We will discuss more on these applications in the upcoming sections.

8.1.2 Sequential pattern mining

This work is motivated by work on sequential pattern mining (Rakesh and Srikant., 1995). We identify the top occurring contiguous word patterns from the student answer corpus. Our hypothesis is that, these frequent patterns in general correspond to the correct answer more than

the wrong answer. For computational reasons, we have only looked at contiguous patterns.

The most frequent word patterns in the student answer corpus for each individual question can be considered to be potential key concepts. But these have to be validated by the reference answer and question. Thus these techniques could be coupled with QA approaches to build a complete system.

We already tried this approach as a part of my intern at Xerox, with mining for contiguous word patterns. In the upcoming chapters, we are going to present a modified version of the sequential pattern mining algorithm for ASAG problem.

8.2 Description

Now, we will briefly present the general working framework of common question answering systems, also focusing on the state-of-the-art works on QA.

8.2.1 General framework of QA systems

Given the question and the corpus, we are to find the best possible answer. The main stages involved in the process are,

- Question analysis and classification
- Query constructor
- Document retrieval
- Sentence retrieval and annotation
- Answer extraction and validation

Out of these, sentence annotation and answer extraction are the most relevant stages to ASAG and we will study the relationship in the upcoming section.

8.2.2 Readings

Here, we will briefly present two state of the art works in Question answering,

aNMM: Ranking Short Answer Texts with Attention-Based Neural Matching Model (Yang Liu, 2016)

- Incorporates attention scheme over question terms to model question's focus

Noise-Contrastive Estimation for Answer Selection with Deep Neural Networks (Rao Jin-feng and Lin., 2016)

- Semantic distance between representations of question and answer
- Candidate sentences are ranked in the decreasing probability of containing the answer
- Pairwise ranking approach to learn relative order of answer pairs, minimizing the total number of inversions in the rankings

8.3 Relevance to ASAG

Building on the idea of key concepts and their relevance to ASAG, here we would present the aspects of QA systems that could be helpful to ASAG systems. We would like to directly extract key concepts by just using the question, answer corpus and later validate using reference answer.

8.3.1 Answer sentence selection

This is one of the last stages of question answering systems where we compare question with potential answer containing sentences (generated using sentence retrieval). In a short answer grading setup we can just do the same on the student answer corpus to either identify possible correct sentences.

In our computer science dataset (CSD) or in general any ASAG setting, each student answer is short and only contains a couple of sentences. Each of these potentially contains the answer, thus identify the sentences as in QA is not required. It might be a better idea to identify potential correct phrases than sentences.

8.3.2 Sentence annotation

After selecting the potential answer containing sentences, each of these sentences are annotated possibly in different ways. They are,¹

- Named Entity Recognition (NER)
- Dependency parsing
- Noun phrase chunking
- Semantic role labeling

Each of these could be chosen depending on the requirement of the current QA setting.

8.3.3 Answer extraction

After the sentence annotation, question type and the annotations could be utilized to identify the possible answer by appropriate matching.

These last two stages, sentence annotation and answer extraction are very relevant to the task of ASAG in identifying key concepts. Since all the sentences in the student answer corpus are potential answer containing sentences, answer sentence selection wouldn't be of much use.

¹<https://hpi.de/naumann/teaching/teaching/ws-1112/question-answering.html>

Chapter 9

A New ASAG system

In this chapter we will present a new short answer grading system, developed using the concepts and ideas built in the past chapters.

9.1 Pattern vectors

In Chapter 7, we have looked at different kinds of word embeddings and their relevance to the task of short answer grading. We also presented different vector based similarity measures to compare the average vectors computed for each answer. Later in Chapter 8 we explored sequential pattern mining and discussed on its role of key concepts from the answer corpus.

In this chapter we combine the aspects of word embeddings and sequential pattern mining by presenting a new concept of pattern vectors. This is inspired from the performance of our new baseline, where we represented each answer (student and reference) with a vector computed as average of all the word vectors from the answer. Since in the context of pattern mining we consider word patterns instead of the entire answer, we are going to compute a similar vector for each pattern (average). For the future usage of the word pattern we use the pattern vector to capture the semantic and contextual information.

The sequential pattern technique utilizing the pattern vectors would be covered in the upcoming sections.

9.2 Utilizing the question

The short answer grading task differs from a similarity task in it's need to evaluate the answer given a particular question. In Chapter 8, we have explored the techniques used in answer sentence selection step of QA systems to find key concepts but these haven't produced significant results. This is because of the different in the nature of tasks of question answering and short answer grading. In this Chapter, we try to explore this as a separate component of the evaluation system and see it's performance in the relevance measure.

9.3 Support

As discussed earlier, we will be combining the approach of pattern mining with utilizing word embeddings in the form of pattern vectors. Earlier, the support of a particular word pattern (p) in computed as follows,

$$support(p) = \frac{\sum_{i,p \in A_i} 1}{\sum_i 1}$$

i.e fraction of answers than contain this particular pattern.

Here, we propose different ways of computing support for a pattern vector using the pattern vectors from the other answers. Let's assume that the pattern under consideration belongs to answer i .

1. Finding the average of cosine similarity with each of the answer vectors

$$support(p) = \frac{\sum_{j=1, j \neq i}^n cosine_similarity(p, AV_j)}{n - 1}$$

2. Finding the average over all other pattern vectors

$$support(p) = \frac{\sum_{j=1, j \neq i}^n \frac{\sum_{q \in AV_j} cosine_similarity(p, q)}{(|q \in AV_j|)}}{n - 1}$$

3. Average of cosine similarity with best matching pattern in each other answer

$$support(p) = \frac{\sum_{j=1, j \neq i}^n \max_{q \in AV_j} (cosine_similarity(p, q))}{n - 1}$$

Of the three measures, third ones seems to be best measure because the pattern length in general very small compared to the answer length and finding cosine similarity between two such vectors doesn't seem reliable. The meaning they capture is also very different because the answer vector could convey the entire meaning.

Note that the support is computed just using the student corpus and neither of question or reference answer are used. These help in capturing the variations present in student answer corpus. We will see a more detailed analysis in the results section.

9.4 Relevance

This measure is in complete contrast to the support and it only uses the reference answer and question to compute relevance for each pattern vector. This primarily includes two parts, one is the relevance w.r.t the question and the other w.r.t the reference answer.

1. **Question:** For every pattern we find it's best matching with the question in lines similar to the third equation in support computation,

$$relevance_question(p) = \max_{q \in Q} (cosine_similarity(p, q))$$

2. **Reference answer:** For every pattern we find it's best matching with the reference answer,

$$relevance_refanswer(p) = \max_{q \in RA} (cosine_similarity(p, q))$$

Given a pattern it belongs to one of these categories,

- Part of correct answer
- Part of wrong answer
- Part of the question itself
- Redundant

The relevance computation with reference answer validates if the pattern indeed belongs to the correct answer. Whereas the relevance computation with question helps in distinguishing non-redundant from redundant.

9.5 Case Studies

We conducted few preliminary experiments to understand the working of our pattern vectors and importance of support and relevance. For each question, we generated three sets of patterns as follows (top 10% in all the three cases),

- Frequently occurring patterns in the student corpus (**F**)
- Best matching patterns with reference answer (**R**)
- Best matching patterns with question (**Q**)

In many cases there are commonalities between the sets **F** and **R**, which validates our hypothesis that frequently occurring patterns correspond to correct answer. But the set **Q** seems very distinct from the sets **F** and **R**. Following are some instances from our dataset reflecting our observations,

Question: What is typically included in a class definition?

Reference answer: Data members attributes and member functions.

F: {'data', 'functions', 'members', 'member', ...}

R: {'functions', 'members', 'member', 'attributes', 'data', ...}

Q: {'usually', 'includes', 'definitions', 'usually contains', ...}

Question: What is the role of a prototype program in problem solving?

Reference answer: To simulate the behavior of portions of the desired software product.

F: {'software', 'product', 'desired', 'desired software', 'desired software product', ...}

R: {'simulate', 'product', 'desired', 'portions', 'software', 'company software', ...}

Q: {'solve', 'problems', 'programs', 'problems solved', 'may solve', 'problems solved sometimes', ...}

Question: What stages in the software life cycle are influenced by the testing stage?

Reference answer: The testing stage can influence both the coding stage phase 5 and the solution refinement stage phase 7.

F: {'refining', 'maintenance', 'production', 'coding', 'construction', 'elaboration', ...}

R: {'solution', 'coding', '5', 'influence', 'phase', '7', '6', '4', '8', ...}

Q: {'test', 'requirements test', 'specification test', 'tested', 'test well', 'program requirements

test', 'back specification test', ...}

Question: Where are variables declared in a C program?

Reference answer: Variables can be declared anywhere in a program. They can be declared inside a function, local variables or outside the functions, global variables.

F: {'function', 'main', 'used', 'beginning', 'global', 'inside', 'anywhere', ...}

R: {'global', 'functions', 'local', 'inside', 'function', 'outside', 'anywhere', 'almost anywhere', ...}

Q: {'declare', 'declare beginning', 'practice declare', 'anywhere declare', ...}

Question: What is the main difference between a while and a do while statement?

Reference answer: The block inside a do while statement will execute at least once.

F: {'loop', 'always', 'condition', 'least', 'executes', 'first', ...}

R: {'block', 'least', 'execute', 'least one', 'always execute', ...}

Q: {'statements', 'differance', 'statements within', 'statements possibility', 'differance statements', ...}

Question: How many constructors can be created for a class?

Reference answer: Unlimited number.

F: {'one', 'different', 'number', 'want', ...}

R: {'unlimited', 'number', 'unlimited long', 'theoretically unlimited', ...}

Q: {'create', 'constructor', 'several', 'one constructor', ...}

This is because direct matching similar to the case of question answering is not the right approach for short answer grading task. Unlike QA, in ASAG the questions are in general lexically very different from the answer and thus leading to low matching. Question tokens present in answer are ignored but still the relevance seems very low. One possible way to utilize question tokens is to look at the context of these tokens in answer corpus which can give some insights on non-redundant parts of answer.

9.6 Results

For our setting, we have use the average of support and relevance as the score for each pattern and the scores of each patterns are averaged to get the scores for the student answers. Computing support by trying to find the best match for the pattern in each of the other reference answers is computationally expensive and hence we have only considered upto 4-grams with this implementation of support. But the general support algorithm and combined approach uses computes support by directly matching the pattern with each other student answer.

We can see the performance of combined approach is similar to the baseline algorithm we developed in Chapter 7 with the RMSE score being slightly better for the baseline. The results for relevance and support individually aren't promising either. The better performance of baseline over relevance means that considering the patterns instead of the entire answer isn't improving the performance but actually declining it a bit.

We also present the confusion matrix for our baseline algorithm, with the scores grouped into six buckets as follows,

TABLE 9.1: Pearson r and RMSE values for different algorithms on CSD dataset

Algorithm	Pearson r	RMSE
Baseline	0.50	1.76
Relevance	0.46	1.92
Support	0.44	2.36
Combined	0.50	2.09
Combined (4-grams)	0.46	2.02

TABLE 9.2: Confusion matrix for baseline algorithm on CSD dataset

scores	< 1	1-2	2-3	3-4	4-5	5	
<1	9	15	2	0	0	0	26
1-2	11	24	17	4	0	0	56
2-3	17	73	90	28	4	0	212
3-4	9	83	178	71	9	0	350
4-5	18	66	200	211	29	0	524
5	14	79	290	410	223	116	1132
	78	340	777	724	265	116	2300

Chapter 10

Question based evaluation

In this chapter we are going to present the idea of question based evaluation.

10.1 Inspiration

We have tried this idea earlier during the intern as we have observed differences in performances of similarity measures for questions based on the question's category. We had looked at different word based lexical, syntactic and semantic measures and developed a system to choose the appropriate algorithm given the category of question.

While reading literature on Question answering systems especially those relating to the task of sentence annotation and answer extraction, we realized the need for considering question's category. For sentence annotation, we can use techniques like dependency parsing and the annotated structures of question and potential answer sentences could then be used to extract answers.

In the context of short answer grading, the syntactic structures of question and sentences in student answer are very different. We observed it on the computer science dataset. But looking at the structures of question and reference answer, there we some relations or pointers for some categories like comparison. In the intern work, we had actually developed a special algorithm for comparison questions using the Stanford typed dependencies.

In the following section we briefly describe the question taxonomy and then we present the results on our dataset category-wise.

10.2 Graesser's taxonomy

Graesser et al. Arthur C Graesser and Huber., 1992; Graesser and Person., 1994 proposed a utterance taxonomy for categorization of questions in an educational setting into 16 categories. This categorization is based on semantic, conceptual and pragmatic criteria instead of lexical and syntactic criteria. In relevance to our dataset we have condensed and selected 7 categories namely, comparison, definition, description, disjunctive, opinion, procedural and specific. Note that these categories are conceptual based and hence different from categories in the question answering.

An example for each category,

- **Comparison:** What is the difference between a constructor and a function?
- **Definition:** What is a linked list?

- **Description:** What is the role of a header-file?
- **Disjunctive:** Using an index outside the bounds of the array generates an error. Is this a compilation error or a run-time error?
- **Opinion:** In one sentence, what is the main idea implemented by insertion sort?
- **Procedural:** How are arrays passed to functions?
- **Specific:** How many constructors can be created for a class?

We can clearly see the differences in requirements for each question category and thus, a category specific algorithm are required.

10.3 Category based computation

Below we present the category-wise results for our support-relevance based computation,

TABLE 10.1: Pearson's r scores across categories on CSD

Category	no. of qs	baseline	relevance	support	combined	combined (4-grams)
comparison	16	0.45	0.40	0.31	0.44	0.41
definition	16	0.42	0.38	0.46	0.45	0.39
description	27(30)	0.57	0.53	0.52	0.57	0.53
disjunctive	0(3)	0.0	0.0	0.0	0.0	0.0
opinion	2	0.10	0.24	0.21	0.36	0.35
procedural	18	0.58	0.54	0.48	0.56	0.53
specific	2	0.21	0.05	0.13	0.10	0.06
overall	81(87)	0.50	0.46	0.44	0.50	0.46
overall RMSE	81(87)	1.76	1.92	2.37	2.10	2.02

10.4 Case Studies

We can observe the differences in the performances of various algorithms across categories. Comparison, definition, description and procedural are the major groups of which the performance isn't good both in comparison and definition based questions. In general, comparison are difficult to handle owing to their nature but dependency parsing could help.

Below we present the set the questions for which the Pearson's r is comparatively low (using the combined approach),

COMPARISON:

Question: What is the difference between a data member and a local variable inside a member function?

Reference answer: Data members can be accessed from any member functions inside the class

definition. Local variables can only be accessed inside the member function that defines them.

Pearson's r: 0.18

Question: What is the main advantage of linked lists over arrays?

Reference answer: The linked lists can be of variable length.

Pearson's r: 0.10

Question: What is the main advantage of a doubly-linked list over a basic linked list?

Reference answer: All the deletion and insertion operations can be performed in constant time, including those operations performed before a given location in the list or at the end of the list.

Pearson's r: 0.20

Question: What is the difference between an array that is declared as static and one that is not?

Reference answer: The static arrays are initialized only once when the function is called.

Pearson's r: 0.27

Question: What is the advantage of linked lists over arrays?

Reference answer: Linked lists are dynamic structures, which allow for a variable number of elements to be stored.

Pearson's r: 0.03

In general the response for a comparison question consists of listings of various attributes for both the entities that are to be compared. But in some of these above mentioned questions, we can see only the attributes of one of these entities are discussed in the reference answer. Student answers would be different and this could be a cause for its low performance in these questions. Because of the inbuilt structure, comparison questions are difficult to handle with our measures which considers the text as a whole.

DEFINITION

Question: What is a queue?

Reference answer: A data structure that can store elements, which has the property that the last item added will be the last to be removed (or first-in-first-out).

Pearson's r: 0.08

Question: What is a leaf?

Reference answer: A node that has no children.

Pearson's r: 0.31

Question: What is a pointer?

Reference answer: The address of a location in memory.

Pearson's r: 0.17

There could be a lot of variance in the student responses for this category and thus matching with a single reference answer will not be of significant help. This can be clearly seen the massive difference between the performance of support and relevance for definition questions (in table 8) with average Pearson's r being 0.38 with relevance whereas 0.46 with support. For these three questions, though the performance was better with support algorithm, still the overall values are low.

DESCRIPTION

Question: What is the number of operations for insertion sort under a best-case scenario, and what is the best-case scenario?

Reference answer: N (the length of the array) operations achieved for a sorted array.

Pearson's r: 0.12

Question: What are the main operations associated with a stack?

Reference answer: push and pop

Pearson's r: -0.04

Overall the performance of our algorithm is relatively good on description questions, except for these two questions. The answers of the first question were primarily expressed in mathematical symbols, which can vary among answers and thus leading to low performance.

The poor Pearson's r score on the second question is because all except one human annotated scores are 5.0 (one being 4.5) whereas our algorithm linearly interpolates the scores between 0-5 depending on the amount of match. The RMSE value for this question is comparable to other description questions, but correlation coefficient fails to capture the performance. This could either be the problem of our interpolation technique or with the correlation measure.

PROCEDURAL

Question: How can an array be addressed in pointer/offset notation?

Reference answer: By initializing a pointer to point to the first element of the array, and then incrementing this pointer with the index of the array element.

Pearson's r: 0.15

Question: How can you implement a queue with a list?

Reference answer: Keep the rear of the queue pointing to the tail of the linked list, so the enqueue operation is done at the end of the list, and keep the front of the queue pointing to the head of the linked list, so the dequeue operation is done at the beginning of the list.

Pearson's r: 0.31

Question: Briefly, how does selection sort work?

Reference answer: It selects the minimum from an array and places it on the first position, then it selects the minimum from the rest of the array and places it on the second position, and so forth.

Pearson's r: 0.30

Similar to the case of description questions, the overall performance on procedural questions is very good with an average Pearson's r of 0.56. The poor performance in the first instance is because most of the student answers were expressed in mathematical forms, which can potentially vary among student responses. Thus both relevance and support are unable to capture this.

SPECIFIC

Question: How many constructors can be created for a class?

Reference answer: Unlimited number.

Pearson's r: -0.16

This category comprises of only 2 questions in the entire dataset. This particular question had a lot of variance in the answer corpus. The meaning of 'unlimited' was expressed in many different ways as in 'no limit', 'any number you want', 'as many as you want', 'as many as you need', 'several', Thus, our algorithms, relevance and support are unable to capture this leading to poor performance.

10.5 Conclusion

With this new introduction of question categories, we also need to build an automatic system to categorize the questions also using the reference answer. This is needed to make the overall technique automated. In relevance computation we are only considering contribution from reference answer but need to figure out a way to incorporate question as well.

In next chapter we will look at dependency based word embeddings that could help us in capturing the dependency based contextual information for words.

Chapter 11

Dependency based Word embeddings

In Chapter 7, we looked at two kinds of word embeddings namely, GloVe and word2vec. In this Chapter we will look dependency based word embeddings.

11.1 Introduction

The idea of dependency based word representations is from the work by Omer and Goldberg., 2014.

11.1.1 Paper: 'Dependency based word embeddings

Prior works on word embeddings have based on linear contexts and this work focuses on moving to arbitrary contexts. They work on syntactic contexts that are derived from automatically produced dependency parse trees. Word2vec (Mikolov, 2013) is based on negative skip-gram with negative sampling and is based on linear BoW based contexts.

BoW based models capture the topical similarity but in a fixed range around the word whereas these word embeddings capture the functional similarity using the dependency relationships. Prior methods are found to capture relatedness rather than similarity that is captured by these embeddings. Following are some examples signifying the differences between these word embeddings with prior works like word2vec,

Florida

BoW: gainesville, fla, jacksonville, tampa, lauderdale

Depd: texas, louisiana, georgia, california, carolina

Hogwarts

BoW: dumbledore, hallows, half-blood, malfoy, snape

Depd: sunnydale, collinwood, calarts, greendale, millfield

From above examples, we can see that the words closest to a given word are meronyms with linear BoW contexts whereas they are co-hyponymns with dependency based contexts.

11.2 Relevance to ASAG

We have looked at dependency parsing based embeddings because of the need of techniques in sentence annotation, answer extraction stages of question answering.

Earlier we have also used Stanford typed dependencies to build a similarity measure (in chapter 5) along with the UNL measure. But the performance wasn't promising.

11.2.1 Results

We have used the pre-trained word embeddings provided by the authors¹, which are trained on English Wikipedia. Similar to word2vec these are 300 dimensional continuous vector representations.

We have utilized these word embeddings along with word2vec to build an overall measure including support and relevance. We found the performance is better with utilizing dependency based context word embeddings for support computation whereas BoW context based embeddings for relevance computation.

The results are as follows:

TABLE 11.1: Pearson's r scores across categories using depd. vectors on CSD

Category	no. of qs	baseline	relevance	support	combined	combined (word2vec)
comparison	16	0.36	0.36	0.27	0.37	0.44
definition	16	0.48	0.46	0.43	0.49	0.45
description	27(30)	0.53	0.54	0.48	0.56	0.57
disjunctive	0(3)	0.0	0.0	0.0	0.0	0.0
opinion	2	0.27	0.26	0.34	0.35	0.36
procedural	18	0.55	0.54	0.43	0.53	0.56
specific	2	0.36	0.13	0.15	0.15	0.10
overall	81(87)	0.48	0.47	0.40	0.49	0.50
overall RMSE	81(87)	1.10	1.29	1.49	1.35	2.10

Here, we can see an improvement in overall pearson r score albeit by a small margin. There is a clear increase in performance of definition questions from 0.44 to 0.49 but comparison questions have been effected in a negative way. Even the RMSE score of 1.10 is the least of all the methods we have worked on. Thus, a right combination of dependency based vectors and word2vec depending on the category could help us in further improving the performance.

We thus tried using dependency based vectors for support computation whereas word2vec vectors for relevance computation. We obtained the following results,

11.3 Case Studies

As we have a category wise analysis in previous chapter, here we focused on determining on the reasons for improvement in each category. From the above table 11.1 comparing the last two columns we can see significant difference in the categories of comparison and definition. Below, we will analyze the possible reasons for the same.

¹<https://levyomer.wordpress.com/2014/04/25/dependency-based-word-embeddings/>

TABLE 11.2: Pearson's r scores across categories using BoW & depd. vectors on CSD

Category	no. of qs	Pearson's r
comparison	16	0.45
definition	16	0.47
description	27(30)	0.58
disjunctive	0(3)	0.0
opinion	2	0.44
procedural	18	0.55
specific	2	0.18
overall	81(87)	0.51
overall RMSE	81(87)	1.64

COMPARISON

Question: What is the main difference between strings declared using the type string versus strings declared using an array of characters?

Reference answer: The strings declared using an array of characters have a null element added at the end of the array.

Pearson's r (Depd.): 0.23

Pearson's r (BoW): 0.43

Question: What is the difference between an array declared as static, and one that is not?

Reference answer: The arrays declared as static live throughout the life of the program; that is, they are initialized only once, when the function that declares the array it is first called.

Pearson's r (Depd.): 0.38

Pearson's r (BoW): 0.63

Question: What is the main advantage of a doubly-linked list over a basic linked list?

Reference answer: All the deletion and insertion operations can be performed in constant time, including those operations performed before a given location in the list or at the end of the list.

Pearson's r (Depd.): -0.01

Pearson's r (BoW): 0.20

Question: Which implementation (array-based vs. list-based) is preferred, and why?

Reference answer: Link-based, because they are dynamic (no size constraints)

Pearson's r (Depd.): 0.19

Pearson's r (BoW): 0.41

DEFINITION

Question: What is a queue?

Reference answer: A data structure that can store elements, which has the property that the last item added will be the last to be removed (or first-in-first-out).

Pearson's r (Depd.): 0.36

Pearson's r (BoW): 0.08

Question: What is a recursive function?

Reference answer: A function that calls itself.

Pearson's r (Depd.): 0.64

Pearson's r (BoW): 0.38

As we can see in the examples given above, performance is significantly better using BoW contexts for comparison whereas the opposite for definition questions. Although we have presented the examples as above, there are many cases the reverse scenario is observed that is BoW performing better in definition and Depd. performing better in comparison albeit by a small margin. These patterns also didn't seem to be directly related with the diversity of student answers.

11.4 Inference

Looking upon the results and case studies we presented in the earlier sections, we see the need to do a in-depth analysis on the qualitative performance of different word embeddings (GloVe, word2vec, Dependency) on our dataset (especially the vocabulary). Though the performance of word2vec and dependency based embeddings are very similar, a qualitative study would help in determining the requirements of our task from word representations. This could be something like trying to find the closest words to the average vector representing each pattern or an answer.

Chapter 12

Conclusion & Future Work

We started the work on short answer grading by looking at various simple lexical, syntactic and semantic similarity measures. Then through the adaptation of sequential pattern mining we simulated a procedure to identify key concepts. Through some experimentations we figured out the need to categorize the question before designing an algorithm to automatically grade the student answers.

In the early stages of the project we explored deeper into the task of similarity and also tried relating paraphrasing and textual entailment to short answer grading. Realizing the importance of different lexical, syntactic and semantic measures we studied Universal Networking language that potentially captures the syntax and semantic by identifying the semantic structure of the sentences.

We then developed a similarity module using BLEU, JCN, Dependency relations and UNL. The results weren't good with the best pearson r score still below 0.5.

Later, we moved onto the use of word embeddings and the performance was better than all our prior approaches and we had a new baseline of 0.50. We then developed a new framework consisting of support and relevance to separately capture the requirements of a short answer grading task. We moved onto the categorization of question and observed significant differences in the performances across with low performances in categories like Comparison and Definition. We then tried a recent development, dependency based word embeddings and observed a slight increase in performance in the critical categories.

Following are some of the things we can try to improve the system,

- Qualitative study of the performance of different word embeddings on the ASAG task.
- Though we tried different ways of finding the relevance of the answer w.r.t the question, we haven't found a satisfactory way of quantifying it. This is different from a general QA setting and we might probably need some domain knowledge along with the student answer corpus to help us in identifying this.
- Since conceptual question categorization is an important aspect to our system, we need to build an automatic system for the same.
- Will have to try our system on other datasets to improve it's robustness and also helps us in building a question categorization module.

Bibliography

- Androutsopoulos, Ion and Prodrornos Malakasiotis. (2010). "A survey of paraphrasing and textual entailment methods." In: *Journal of Artificial Intelligence Research* 38 (2010): 135-187.
- Arthur C Graesser, Natalie Person and John Huber. (1992). "Mechanisms that generate questions." In: *Questions and information systems*, pages 167-187.
- Burrows Steven, Iryna Gurevych and Benno Stein. (2015). "The eras and trends of automatic short answer grading." In: *International Journal of Artificial Intelligence in Education* 25.1 (2015): 60-117.
- Dan, Avishek and Pushpak Bhattacharyya. (2013). "CFILT-CORE: Semantic Textual Similarity using Universal Networking." In: *Atlanta, Georgia, USA (2013)*: 216.
- Gautam, Shubham and Pushpak Bhattacharyya. (2014). "LAYERED: Description of Metric for Machine Translation Evaluation." In: *WMT 14 Metrics Task, 9th Workshop on Statistical Machine Translation (WMT14)*, Baltimore, USA, 26-27 June, 2014.
- Graesser, A. C. and N. K. Person. (1994). "Question asking during tutoring." In: *American Educational Research Journal*, 31:104-137+.
- Jiang, J. and D. Conrath. (1997). "Semantic similarity based on corpus statistics and lexical taxonomy." In: *In Proceedings of the International Conference on Research in Computational Linguistics, Taiwan*.
- Klein, Dan and Christopher D. Manning. (2003). "Accurate Unlexicalized Parsing." In: *Proceedings of the 41st Meeting of the Association for Computational Linguistics*, pp. 423-430.
- Kong Leilei, et al. (2014). "Detecting High Obfuscation Plagiarism: Exploring Multi-Features Fusion via Machine Learning." In: *International Journal of u-and e-Service, Science and Technology* 7.4 (2014): 385-396.
- Leacock, C. and M. Chodorow. (1998). "Combining local context and WordNet sense similarity for word sense identification. In WordNet, An Electronic Lexical Database." In: *The MIT Press*.
- Mikolov Tomas, et al. (2013). "Distributed representations of words and phrases and their compositionality." In: *Advances in neural information processing systems*.
- Mohler, Michael and Rada Mihalcea. (2009). "Text-to-text semantic similarity for automatic short answer grading." In: *Proceedings of the 12th Conference of the European Chapter of the Association for Computational Linguistics*.
- Mohler Michael, Razvan Bunescu and Rada Mihalcea. (2011). "Learning to grade short answer questions using semantic similarity measures and dependency graph alignments." In: *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies-Volume 1*.
- Omer, Levy and Yoav Goldberg. (2014). "Dependency-Based Word Embeddings." In: *ACL* (2).
- Papineni Kishore, et al. (2002). "BLEU: a method for automatic evaluation of machine translation." In: *Proceedings of the 40th annual meeting on association for computational linguistics*.

- Pennington, Jeffrey, Richard Socher, and Christopher D. Manning. (2014). "GloVe: Global Vectors for Word Representation." In: *EMNLP. Vol. 14*.
- Quirk C., C. Brockett and W. B. Dolan. (2004). "Monolingual Machine Translation for Paraphrase Generation." In: *In Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing, Barcelona Spain*.
- Rakesh, Agrawal and Ramakrishnan Srikant. (1995). "Mining sequential patterns." In: *Proceedings of the Eleventh International Conference on Data Engineering, IEEE*.
- Rao Jinfeng, Hua He and Jimmy Lin. (2016). "Noise-Contrastive Estimation for Answer Selection with Deep Neural Networks." In: *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management. ACM*.
- Resnik., P. (1995). "Using information content to evaluate semantic similarity." In: *In Proceedings of the 14th International Joint Conference on Artificial Intelligence, Montreal, Canada*.
- Roy Shourya, Y. Narahari and Om D. Deshmukh. (2015). "A Perspective on Computer Assisted Assessment Techniques for Short Free-Text Answers." In: *International Computer Assisted Assessment Conference. Springer International Publishing*.
- Stamatatos, Efstathios. (2011). "Plagiarism detection using stopword n-grams." In: *Journal of the American Society for Information Science and Technology* 62.12 (2011): 2512-2527.
- Wu, Z. and M. Palmer. (1994). "Verb semantics and lexical selection." In: *In Proceedings of the 32nd Annual Meeting of the Association for Computational Linguistics, Las Cruces, New Mexico*.
- Yang Liu Ai Qingyao, Guo Jiafeng Croft W. Bruce. (2016). "aNMM: Ranking Short Answer Texts with Attention-Based Neural Matching Model." In: *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management. ACM*.